Trabajo Práctico 2 – Procesos

SISTEMAS OPERATIVOS

Índice

1. Estados y observación de procesos **3**
1.1. Comando `top` 3

2. El sistema de archivos `/proc` **3**

3. PID y proceso padre **4**

4. Creación de procesos: `fork()` y `exec()` **5**

5. Hilos POSIX **5**

1. Estados y observación de procesos

Un proceso es un programa que está corriendo. Cada proceso tiene un identificador único, llamado PID, y normalmente mantiene una relación con el proceso que lo creó. El identificador de ese proceso padre se llama PPID.

Con `ps` ("Process Snapshot") podés ver procesos. Sin opciones muestra los procesos asociados a la terminal actual:

```
$ ps
```

Cada vez que ejecutás `ps`, el shell crea un proceso nuevo, por eso su PID cambia. El shell, en cambio, sigue corriendo y conserva su PID.

Para ver más información y listar procesos de todos los usuarios, probá:

```
$ ps aux
$ ps -fu "$USER"
$ pstree -p
```

La opción `f` muestra las relaciones padre-hijo como un árbol. Con `-T -p PID` podés ver los hilos de un proceso:

```
$ ps -T -p 1234
```

INFO

Consultá las opciones disponibles con `man ps`. Para salir de los visores interactivos, apretá `q`.

1.1. Comando `top`

Con `top` podés ver los procesos ordenados por consumo de recursos. La lista se actualiza periódicamente y el orden puede cambiar porque el planificador selecciona procesos según su prioridad dinámica.

```
$ top
$ top -H
```

La opción `-H` muestra los hilos. También podés usar `htop`, que ofrece una interfaz interactiva más cómoda.

2. El sistema de archivos `/proc`

`/proc` es un sistema de archivos virtual que genera el núcleo. No representa un dispositivo de almacenamiento: sirve para consultar información sobre procesos, memoria, CPU y otras partes del sistema.

Hay un directorio numérico por cada proceso activo. El enlace `/proc/self` apunta al proceso que lo está consultando:

```
$ ls -l /proc/self
lrwxrwxrwx 1 root root 0 ... /proc/self -> 4856
$ ls -l /proc/self
lrwxrwxrwx 1 root root 0 ... /proc/self -> 4857
```

Dentro de cada directorio de proceso hay archivos como `status`. Ahí podés encontrar el estado, el PID, el PPID y los cambios de contexto:

```
$ grep ctxt /proc/self/status
voluntary_ctxt_switches: 0
nonvoluntary_ctxt_switches: 2
```

EXTRA

También podés explorar `/proc/cpuinfo`, `/proc/meminfo` y `/proc/interrupts`. Compará esa información con la salida de los comandos del sistema.

3. PID y proceso padre

POSIX ofrece llamadas al sistema para consultar la identidad de un proceso. Las funciones que vamos a usar son:

```
pid_t getpid(void);
pid_t getppid(void);
```

Creá el archivo `obtenerpid.c` con este contenido:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());
    return 0;
}
```

Compilalo y ejecutalo:

```
$ gcc -Wall -Wextra -o obtenerpid obtenerpid.c
$ ./obtenerpid
PID: 7014
PPID: 3814
$ strace ./obtenerpid
```

EXTRA

Investigá qué hacen las opciones `-Wall` y `-Wextra`. Podés consultar `man gcc` o buscar la documentación online. Después, compilá el programa con y sin esas opciones y compará los mensajes del compilador.

El PPID normalmente corresponde al shell que lanzó el programa. Con `strace` podés ver las llamadas al sistema que hace durante su ejecución.

4. Creación de procesos: `fork()` y `exec()`

`fork()` crea un proceso hijo como copia del proceso que lo invoca. Su valor de retorno te permite distinguir los dos caminos: -1 indica un error, 0 identifica al hijo y un valor positivo identifica al padre.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }
    if (pid == 0)
        printf("Hijo: PID=%d PPID=%d\n", getpid(), getppid());
    else
        printf("Padre: PID=%d PPID=%d\n", getpid(), getppid());
    return 0;
}
```

```
$ gcc -Wall -Wextra -o creaproceso creaproceso.c
$ ./creaproceso
Padre: PID=7094 PPID=3814
Hijo: PID=7095 PPID=7094
$ strace -f ./creaproceso
```

La llamada `exec()` reemplaza el programa que está corriendo por otro. El shell suele crear un hijo con `fork()` y después cargar ahí el comando pedido mediante una variante de `exec()`.

5. Hilos POSIX

Los hilos te permiten ejecutar varias secuencias dentro del mismo proceso. Comparten la memoria del proceso, pero cada uno tiene su propio estado de ejecución.

El siguiente ejemplo crea dos hilos que escriben indefinidamente:

```
#include <pthread.h>
#include <stdio.h>

void *saludar(void *arg) {
    const char *texto = arg;
    while (1) printf("%s\n", texto);
    return NULL;
}

int main(void) {
    pthread_t uno, dos;
    pthread_create(&uno, NULL, saludar, "Hola");
    pthread_create(&dos, NULL, saludar, "mundo");
    pthread_join(uno, NULL);
    pthread_join(dos, NULL);
    return 0;
}
```

Compilalo enlazando la biblioteca POSIX de hilos. Para detenerlo, apretá `Ctrl-C`:

```
$ gcc -Wall -Wextra -pthread -o holahilo holahilo.c
$ ./holahilo
Hola
mundo
...
```

EXTRA

Compará la salida de `ps -T -p PID` antes y después de crear los hilos. Repetí las pruebas con `strace -f` y fijate qué cambia entre procesos e hilos.